Breadth First Search

AlgoExpert · Medium · Graphs

The problem

- Return node names of a tree-like graph level by level.

Intuition

- Breadth-first search visits nodes level by level using a queue.

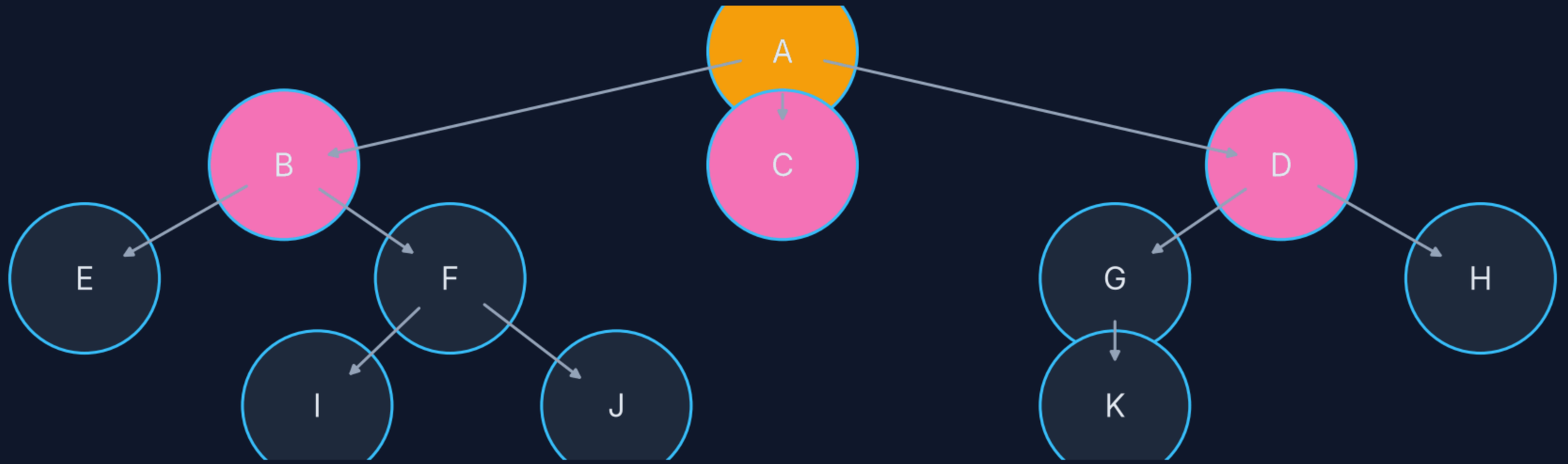
Brute force vs optimal

- Same O(V + E) as DFS — the order is what differs
- Queue (FIFO) instead of stack/recursion
- BFS gives shortest paths in unweighted graphs

Approach

- 1. Enqueue the root
- 2. Pop, record the name, enqueue children
- 3. Repeat until the queue is empty

Worked example

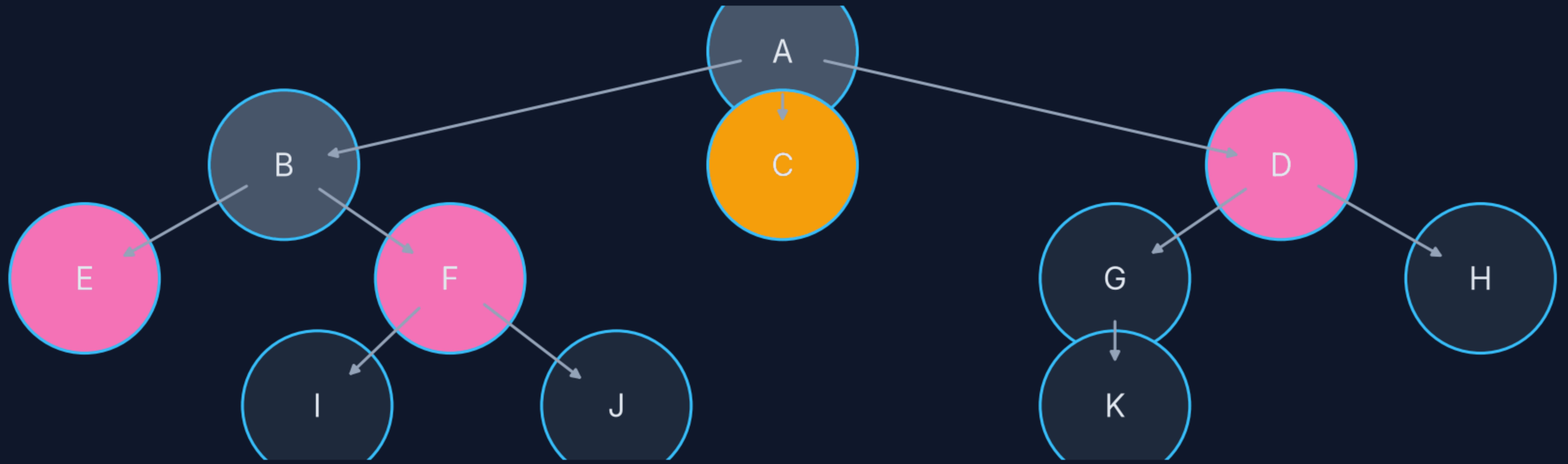


queue (front → back)



Dequeue A; enqueue B, C, D

Worked example

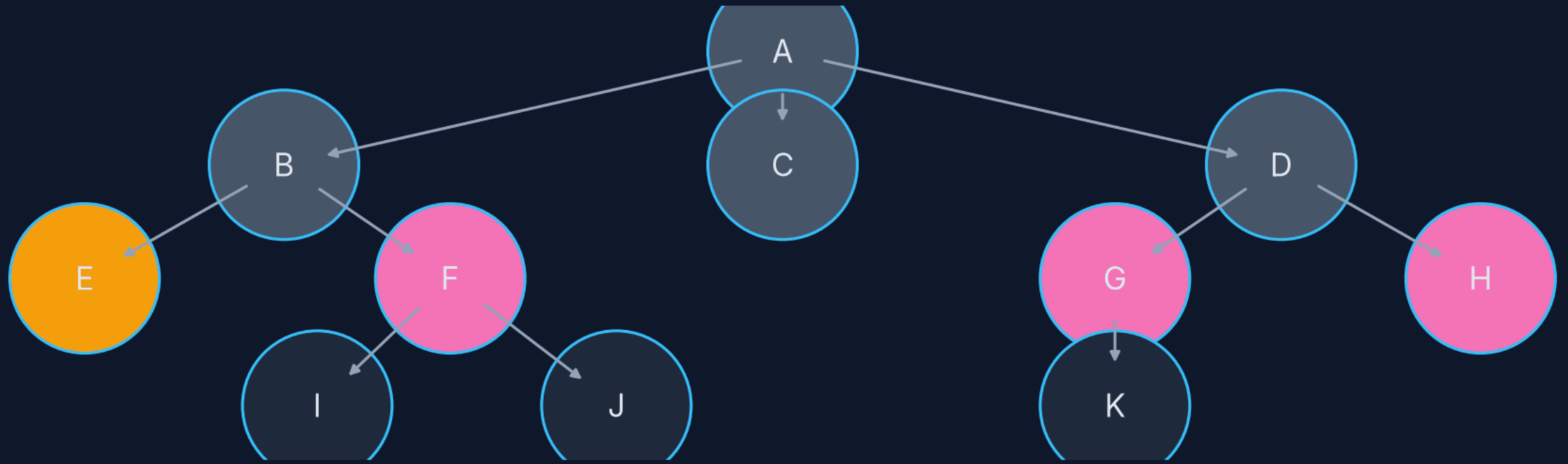


queue (front → back)



Dequeue C; enqueue nothing

Worked example

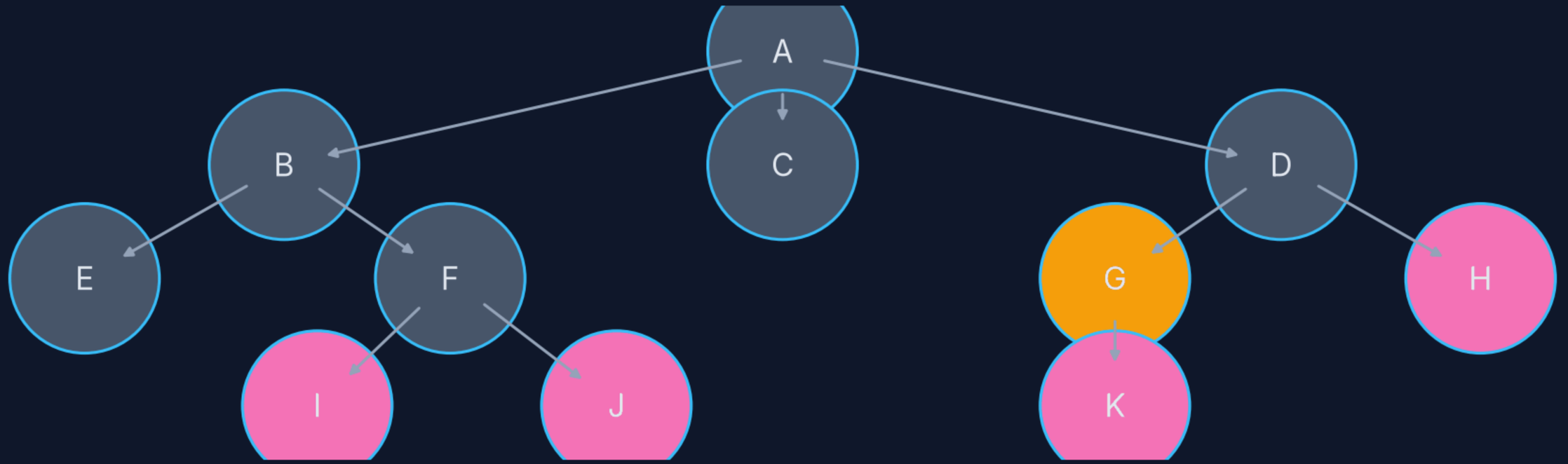


queue (front → back)



Dequeue E; enqueue nothing

Worked example

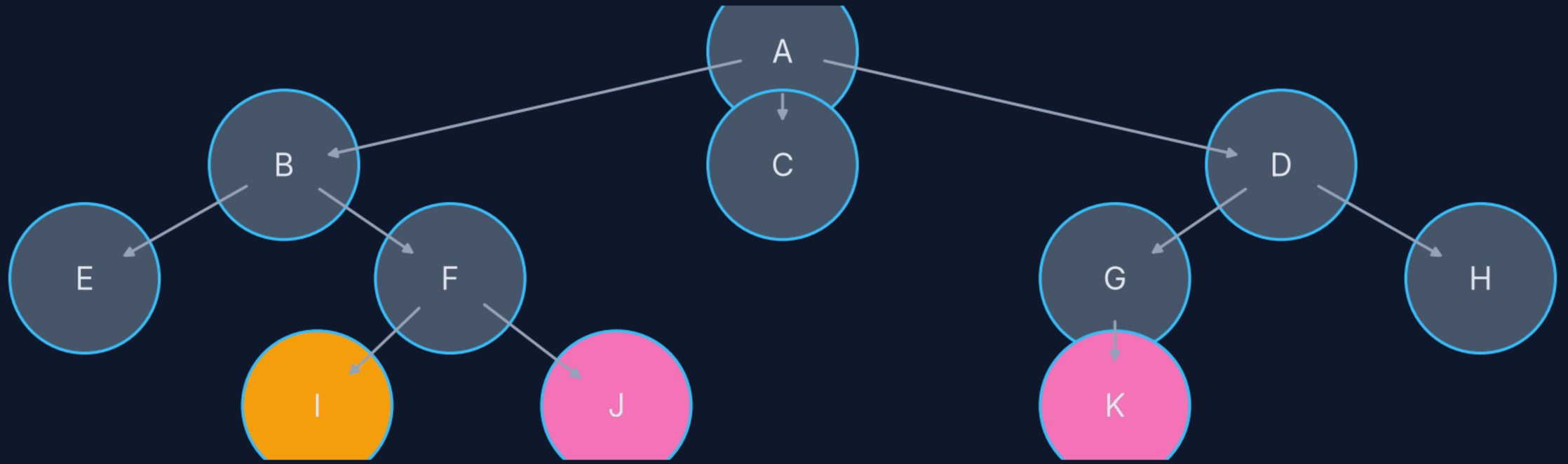


queue (front → back)



Dequeue G; enqueue K

Worked example

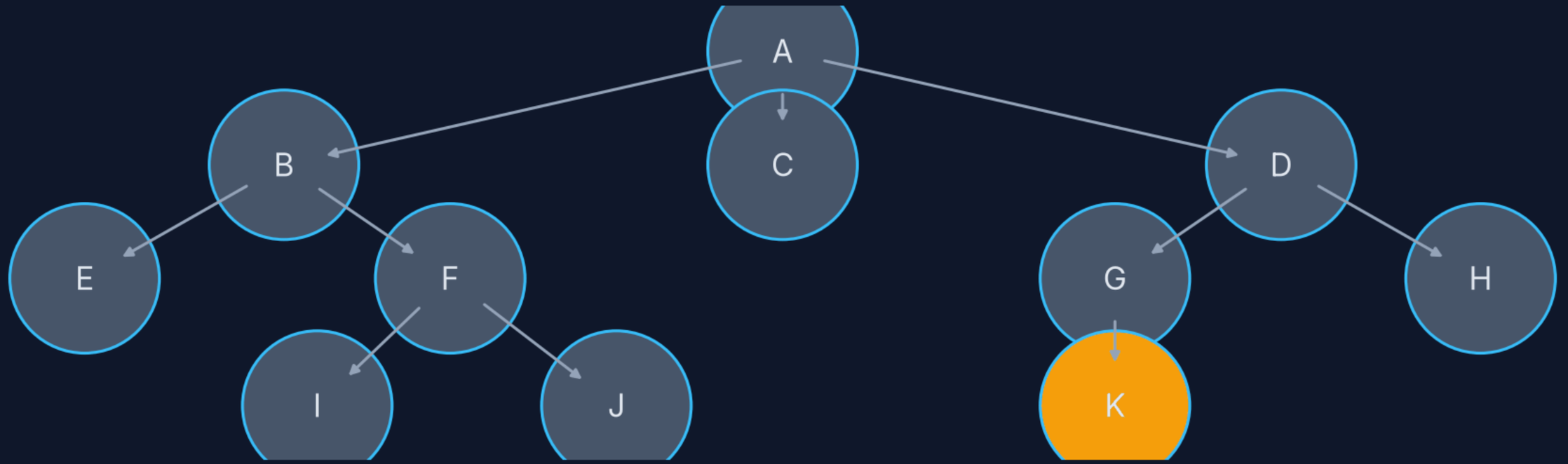


queue (front → back)



Dequeue I; enqueue nothing

Worked example



queue (front → back)



Dequeue K; enqueue nothing

Java solution

```java
public class BreadthFirstSearch {
    public static class Node {
        public final String name;
        public final List<Node> children = new ArrayList<>();
        public Node(String name) { this.name = name; }
        public Node addChild(String n) { children.add(new Node(n)); return this; }

        public List<String> breadthFirstSearch(List<String> out) {
            ArrayDeque<Node> q = new ArrayDeque<>();
            q.add(this);
            while (!q.isEmpty()) { Node c = q.poll(); out.add(c.name); q.addAll(c.children); }
            return out;
        }
    }
}
```

Complexity

- Time: O(v + e)
- Space: O(v)

Edge cases & pitfalls

- Use a real queue (ArrayDeque), not list.remove(0)
- General graphs need a visited set
- Children enqueued left to right

Interview tips & follow-ups

- Follow-up: level-order with level boundaries
- Follow-up: shortest path in a grid

Recap

- Breadth-first search visits nodes level by level using a queue.
- Complexity: O(v + e)